Autonomous Synthesis and Algorithmic Verification of Power Electronic Circuits using Agentic AI

Institution: Indian Institute of Technology Kharagpur (IIT KGP) — Remote Internship
Date: September 2026

Abstract

Designing switch-mode power converters remains a manual, iterative process. Engineers must select a topology, calculate component values, build SPICE netlists, simulate behaviour, inspect waveforms, and tune control loops — a cycle that can take days for complex topologies like LLC resonant converters or Vienna rectifiers.

This project investigates whether AI can automate this workflow end-to-end: accepting a natural-language specification (e.g., *"closed-loop buck converter, 120 V in, 48 V out, 100 kHz"*) and producing a verified, simulation-ready SPICE netlist. Through three iterative phases — direct LLM generation, parameter injection, and finally Agentic AI — we map the boundary between what generative AI can and cannot do in deterministic physical simulation. The final architecture pairs an Agentic AI system with a curated circuit library, FFT-based diagnostics, and an algorithmic optimisation loop that together produce reliable, autonomous converter designs.

Motivation: Why This Project Matters

Modern AI chatbots can generate plausible SPICE netlists in seconds. But plausible is not correct, and in power electronics, the distinction matters. When a language model generates buggy Python, the interpreter throws an exception. When it generates a buggy netlist that is used to build a prototype, the consequences can include thermal runaway from magnetic saturation, semiconductor breakdown from voltage overshoot, or electromagnetic interference from a missing snubber. An answer that *looks right but is wrong* is more dangerous than no answer at all.

General-purpose AI treats SPICE as text to be predicted, not as a physical system to validate. It does not simulate the circuit, check output convergence, or verify phase margin. This project addresses that gap: every netlist is simulated in NgSpice, analysed by FFT-based diagnostics for ripple and stability, and — if unstable — automatically re-tuned by a deterministic optimiser before being presented to the user.

Use Cases

- **Rapid prototyping and design-space exploration.** Engineers designing EV chargers or server power supplies can evaluate multiple topologies across varying specifications in minutes instead of days.
- **Education.** Students can receive verified starting netlists from plain-language descriptions, along with the intermediate calculations that explain *why* each component value was chosen.
- **Pre-silicon IC verification.** Semiconductor companies can auto-generate application circuits from datasheet specification tables, accelerating verification and reducing the risk of publishing incorrect application notes.

---

1. Phase One: Direct LLM Netlist Generation and the Circuit Library

1.1 Initial Hypothesis

The first approach used Google's Gemma-4 model, hosted locally via LM Studio (OpenAI-compatible API on `localhost:1234`). The idea was straightforward: if LLMs can write Python from natural-language descriptions, they should be able to write SPICE — a text-based netlist format that has been in use since the 1970s.

A detailed system prompt specified SPICE syntax rules: component naming prefixes (`R` for resistors, `L` for inductors, `S` for switches, `M` for MOSFETs), `PULSE` and `SIN` source formats, node connectivity requirements, and the distinction between switch and MOSFET models.

1.2 Why It Failed

Gemma-4 produced well-formatted `.cir` files, but the electrical content was consistently wrong:

- **Unrealistic component values.** Assigning 1 H inductors or 1 pF capacitors to a 100 kHz converter — values that would prevent normal switching operation entirely.
- **Topology errors.** Floating nodes (nodes connected to only one terminal, making the conductance matrix singular) and reversed diode polarity.
- **Model confusion.** Persistently mixing voltage-controlled switch syntax (`SW`) with MOSFET syntax (`NMOS`), causing model-type mismatch errors.
- **Arbitrary control-loop gains.** Assigning PI gains (Kp = 1.0, Ki = 100) without any relationship to the plant transfer function, pushing crossover frequencies beyond the switching frequency and causing divergent oscillations.

The root cause is architectural: an autoregressive language model predicts the next token from statistical patterns. It has no internal differential-equation solver, no Laplace-domain evaluator, and no concept of Kirchhoff's laws. It can reproduce the *form* of a SPICE netlist but cannot verify that the result will converge.

1.3 Building the Circuit Library

Rather than trying to teach the LLM physics, we decided to provide it with verified examples it could use as templates. This led to a substantial manual engineering effort: building a comprehensive **Circuit Library** from scratch. Each design was hand-built, simulated, debugged, and validated until its waveforms matched theoretical expectations.

The library covers the following topologies:

DC-DC (Non-Isolated): Buck, Synchronous Buck, Boost, Cuk, SEPIC, and Four-Switch Buck-Boost (FSBB).

DC-DC (Isolated): Flyback, Forward, Push-Pull, Active Clamp Forward, LLC half-bridge resonant, CLLC bidirectional resonant, and Dual Active Bridge (DAB). The CLLC and DAB require phase-shift modulation of two full bridges with resonant tank elements tuned for zero-voltage switching.

AC-DC Rectifiers and PFC: SCR-controlled rectifier, single-phase Boost PFC (switched and averaged), Interleaved PFC, Totem-Pole PFC, Active Front End (AFE) rectifier, Vienna Rectifier (three-phase, three-level, with inner current loop and midpoint balancing), and a single-stage AC-DC LLC. The Vienna rectifier alone required over 100 lines of SPICE behavioural sources.

DC-AC Inverters: Three-phase inverter with sinusoidal PWM and closed-loop voltage regulation.

Each netlist used `.param` statements for key values (e.g., `.param Vin_val=24`, `.param L1_val=1m`), allowing specification changes without modifying the underlying topology. This parameterisation became central to Phase Two.

1.4 The Python-PowerShell Automation Architecture

With the library in place, we built a pipeline to automatically match user prompts to templates, inject them into the LLM context, and run the generated netlist through NgSpice.

Library Manager (`library_manager.py`). Matched user requests to templates via tiered fuzzy scoring: exact substring match scored 100, base-key match scored 50, and bag-of-words fallback scored 40.

```
# library_manager.py
if key in request_lower or re.search(pattern, request_lower):
    score = 100
else:
    base_key = key.replace(" average", "").replace(" real", "")
    if base_key in request_lower:
        score = 50
    elif all(re.search(r'\b' + w + r'\b', request_lower) for w in words):
        score = 40
```

Prompt Engine (`prompt_engine.py`). Injected the matched template as a verified in-context example, instructing Gemma-4 to copy the structure and modify only the numerical values.

```
# prompt_engine.py
ref_example = (
    f"--- Example 3: {label} (from verified library: {filename}) ---\n"
    f"IMPORTANT: This is a VERIFIED working netlist. When the user asks\n"
    f"for a {label}, copy this example exactly and only change the values\n"
    f"the user specifies (voltage, frequency, load, etc.).\n"
    f"{sanitized}\n"
)
system_prompt = _SYSTEM_PROMPT_TEMPLATE.replace("{REFERENCE_EXAMPLE}", ref_example)
```

Netlist Extraction (`pipeline.py`). The LLM's streaming response was scored to determine whether the netlist appeared in the reasoning or the final output, since Gemma-4 frequently embedded valid SPICE lines in its chain-of-thought. A density heuristic counted SPICE-like lines in each stream.

```
# pipeline.py
def _netlist_score(text: str) -> int:
    score = 0
    for line in text.split('\n'):
        stripped = line.strip()
        if re.match(r'^[RLCVIDSMQEFGH]\w*\s+', stripped, re.IGNORECASE):
            score += 2
        elif re.match(r'^\.(tran|ac|dc|model|control|endc|end)\b', stripped, re.IGNORECASE):
            score += 2
    return score
```

Automatic fixers corrected the most common LLM errors: the S/M prefix confusion (switch vs. MOSFET) and unit suffixes appended to model parameters (e.g., `BV=100V` corrected to `BV=100`). The validated netlist was then executed via `ngspice_con.exe` in batch mode, with output parsed and plotted automatically.

1.5 Where It Succeeded and Failed

The example-injection approach was a clear improvement over blind generation. For requests closely matching a template, the model produced structurally correct netlists that simulated without syntax errors.

However, two fundamental problems remained. First, **the model could not adapt component values** for new operating points — it copied values from the template even when the specifications changed, because it had no way to recalculate inductance from volt-second balance or capacitance from ripple constraints. Second, **control-loop gains were almost never correct** for a new operating point. The PI gains were hand-tuned for

specific component values in the template; changing any parameter altered the plant transfer function, rendering the original gains inadequate. The output voltage would oscillate, latch, or collapse — and the model had no feedback mechanism to observe or correct this.

---

2. Phase Two: The Parameter-Injection Architecture

2.1 Core Insight

The LLM was most reliable when it was *not* producing SPICE syntax. It could discuss power electronics competently — it knew the buck converter duty cycle is D = Vout/Vin, that a SEPIC can step up or down — but translating that knowledge into a netlist introduced errors. The new strategy separated calculation from synthesis entirely: the LLM would only compute numerical values, which deterministic Python code would inject into the verified templates.

This restructured the pipeline into three steps: **Calculate → Extract → Inject**.

2.2 The Three-Step Pipeline

Step 1: Component Calculation. The first LLM call was explicitly forbidden from producing SPICE syntax. It was told to show its formulas, state assumptions, and output a clear list of final values.

```
# prompt_engine.py
system_prompt = (
    "You are an expert power electronics engineer. Your task is to "
    "calculate the component values (Inductors, Capacitors, Resistors, "
    "Frequencies, etc.) required for a power converter design.\n\n"
    "RULES:\n"
    "1. Read the user's requirements (e.g. Vin, Vout, Power, Ripple).\n"
    "2. State your assumptions and formulas clearly.\n"
    "3. Output the final calculated values in a clear list at the end.\n"
    "4. Do NOT output a SPICE netlist in this step."
)
```

Step 2: Structured Extraction. A second LLM call converted the free-form calculations into a JSON dictionary mapped to the `.param` keys found in the selected template. The prompt engine automatically read the template's `.param` lines and presented them as the required JSON keys.

```
# pipeline.py
json_match = re.search(
    r'```(?:json)?\s*(\{.*?\})\s*```', json_output, re.DOTALL | re.IGNORECASE
)
if json_match:
    json_clean = json_match.group(1).strip()
else:
    json_fallback = re.search(
        r'(\{\s*"[A-Za-z0-9_]+"\s*:.*?\})', json_output, re.DOTALL
    )
    json_clean = json_fallback.group(1).strip() if json_fallback else json_output
```

Step 3: Deterministic Injection. The parsed JSON was iterated and each value substituted into the template's `.param` statements via regex.

```
# pipeline.py
netlist = raw_reference_content
for key, value in params_dict.items():
    pattern = r'(\.param\s+' + re.escape(key) + r'=).*'
```

```
    replacement = r'\g<1>' + str(value)
    new_netlist = re.sub(pattern, replacement, netlist, flags=re.IGNORECASE)
    if new_netlist != netlist:
        print(f"  Injected {key} = {value}")
    else:
        print(f"  Warning: {key} not found in .param statements")
    netlist = new_netlist
```

2.3 Where It Succeeded

The improvement was immediate. **Syntax errors vanished** — the template was never touched by the LLM, so the S/M confusion, floating nodes, and missing models simply disappeared. **Topology was always correct**, since the fuzzy matcher guaranteed the right converter structure. The pipeline also saved intermediate outputs (`step1_calculations.txt`, `step2_parameters.json`) alongside the final netlist, creating a complete audit trail.

2.4 Where It Failed — The Generalisation Bottleneck

Despite eliminating syntax errors, deeper problems remained.

Unreliable calculations. The LLM could correctly state the inductance formula L = Vout(1 - D) / (2 * dIL * fs) but then substitute values incorrectly, forget unit conversions (kHz to Hz), or apply a boost formula to a buck topology. These errors were non-systematic and varied between runs.

Control-loop gains remained the weakest link. Proper controller design requires computing the open-loop transfer function — a frequency-domain analysis involving the inductance, capacitance, load resistance, and ESR in a topology-specific arrangement. The LLM could not perform this analysis. It would copy gains from internet examples or guess round numbers.

No generalisation across topologies. The `.param` keys differed between templates (`L1_val` is the output inductor in a buck but the power-transfer inductor in a DAB), and each topology used different design equations. Extending the system to a new converter required manual inspection and correction.

No feedback loop. Most critically, the pipeline was open-loop with respect to simulation results. If the PI gains were wrong, the system had no mechanism to detect the instability, diagnose the cause, and re-compute. It could detect NgSpice crashes, but it could not detect convergence to a *physically incorrect result* — a far more insidious failure mode.

These failures made it clear the project needed a fundamentally different kind of AI system — one that could execute, observe, and iterate autonomously.

---

3. Phase Three: The Transition to Agentic AI

3.1 Why an Agent, Not a Better LLM

The Phase 2 failures were not failures of intelligence — Gemma-4 understood power electronics conceptually — but failures of architecture. The pipeline could not close a feedback loop. It could not run a simulation, inspect the waveform, decide the output was oscillating, reason about why, and re-compute the gains. It produced a single, irrevocable answer.

An *agent* is fundamentally different: it receives a goal, plans a sequence of actions, executes each one (running commands, reading files, writing files), observes the result, and decides what to do next. The loop continues until

the goal is achieved. An agent can do something no single-pass LLM can: *try something, see that it failed, and try something different.*

3.2 Why Not Just Let AI Do Everything?

A fair question: if an agent can browse files, execute commands, and iterate on its own — why build any infrastructure at all? Why not just ask it to design a converter?

The answer became clear through experimentation. An agent without domain-specific tools is just an LLM with extra steps. When the agent simulated a boost converter and the output oscillated at 2 kHz with 40 V peak-to-peak, it could read the numbers in `output_data.txt` but had no principled way to determine whether 40 V was acceptable, whether 2 kHz indicated a loop instability or normal switching ripple, or what specific adjustment to Kp or Ki would help. It would guess — halving or doubling a gain — simulate again, get a different but equally wrong result, and continue in a random walk through parameter space that rarely converged.

The agent was, in effect, an engineer who could operate a simulator but had never taken a controls course — access to the equipment, but no judgement about what the readings meant.

3.3 The Antigravity SDK Architecture

The agent was built on the **Google Antigravity SDK**, initialised via a Python script that defined its persona and granted filesystem and command-execution permissions.

```
# agent_pipeline.py
from google.antigravity import Agent, LocalAgentConfig, CapabilitiesConfig

SYSTEM_PROMPT = """You are an expert Power Electronics and SPICE engineering agent.
Your goal is to autonomously design, test, and verify SPICE netlists for
power converters.

Rules:
1. When asked to design a converter, look at the reference netlists in the
  'Circuit Library' directory in the current workspace.
2. Generate the appropriate `.cir` file for the new design.
3. Automatically run ngspice using your command line tools to verify the
  circuit works and meets the requirements.
"""

config = LocalAgentConfig(
   system_instructions=SYSTEM_PROMPT,
   capabilities=CapabilitiesConfig(),
   api_key=api_key,
)
await run_interactive_loop(config)
```

Upon receiving a user request, the agent would: read the Circuit Library, open the relevant template, study its `.param` definitions, calculate new component values, write the modified netlist, execute NgSpice, read the output, and — if the waveform was wrong — diagnose the problem, modify the netlist, and re-simulate. This was the iterative feedback loop that Phases 1 and 2 had lacked.

3.4 Initial Failures

In practice, the early results were disappointing. The agent would iterate through 5 to 15 simulation cycles without converging.

It could not interpret waveform data. Reading a 50,000-row text file of time-voltage pairs, it had no reliable method for determining whether the signal was settling, oscillating with growing amplitude, or oscillating with constant amplitude. A crude average could mask an oscillating signal whose mean happened to be near the target.

Guessing was not a strategy. The agent would reason in prose — *"the output is too high, so I should increase Kp"* — but this reasoning was often wrong. In a closed-loop converter, the relationship between a controller gain and the output voltage is mediated by the entire loop transfer function; qualitative intuition is frequently incorrect.

Manual intervention was needed. During early development, the most effective debugging technique was the most primitive: the human operator would open the NgSpice GUI, take a screenshot of the oscillating waveform, and paste it into the agent's chat. This gave the agent the visual context to reason about the failure — but it completely defeated the purpose of automation.

3.5 Root Cause

The core problem was clear: the agent had the *mechanism* for iteration (tool use, file I/O, command execution) but lacked the *instruments* for evaluation. A human engineer diagnosing instability uses an oscilloscope and a spectrum analyser. The oscillation frequency tells you *what* is wrong: sub-100 Hz suggests integrator windup, hundreds to thousands of hertz suggests LC resonance near the crossover frequency, and switching-frequency ripple suggests insufficient capacitance. The agent needed algorithmic equivalents of these instruments.

---

4. The Automated Diagnostic and Optimisation Toolkit

4.1 The Diagnostic Tools

Three Python scripts were developed to serve as the agent's diagnostic instruments, each callable via a shell command so the agent could invoke it, read the output, and act on the diagnosis.

4.1.1 FFT Diagnostics

The script `automation/fft_diagnostics.py` serves as the agent's oscilloscope and spectrum analyser. It loads the raw `output_data.txt`, discards the first half (startup transient), computes the FFT of the steady-state signal, and reports the dominant oscillation frequency and peak-to-peak ripple. Heuristic rules map frequencies to specific instability types:

```
# fft_diagnostics.py
if peak_to_peak > 2.0 and dom_freq < 100:
   print("Diagnosis: Very low frequency oscillation detected. "
         "This is typically PI integrator windup. Try lowering Ki.")
elif peak_to_peak > 2.0 and dom_freq < 5000:
   print("Diagnosis: Mid-frequency oscillation detected. "
         "This is typically an LC tank resonance. "
         "Try adding RC damping or reducing Kp.")
elif peak_to_peak > 2.0:
   print("Diagnosis: High-frequency oscillation. "
         "Likely just switching ripple. Increase output capacitance.")
else:
   print("Diagnosis: Stable.")
```

These thresholds are grounded in converter physics: LC filter resonance falls in the hundreds-to-low-thousands Hz range, integrator windup manifests sub-100 Hz, and switching ripple occurs at the switching frequency (typically 25-200 kHz). The script also generates dual-panel diagnostic plots (time-domain waveform and frequency spectrum).

4.1.2 SPICE Optimizer

When FFT diagnostics declare instability, the agent invokes `automation/spice_optimizer.py` — a deterministic grid search over the controller gain space. For each candidate (Kp, Ki) pair, it substitutes the values

into the netlist, runs NgSpice, and evaluates the result against a cost function:

Cost = |V_avg - V_target| + 2 x V_ripple(p-p)

The first term penalises steady-state error; the second penalises ripple (weighted 2x because ripple is typically a harder constraint). A 3x3 grid requires nine simulation runs, taking one to five minutes total.

```
# spice_optimizer.py
def evaluate(data_file, target_v):
    data = np.array(data)
    half = len(data) // 2
    v = data[half:, 1]

    avg_v = np.mean(v)
    p2p = np.max(v) - np.min(v)

    cost = abs(avg_v - target_v) + p2p * 2
    return cost, avg_v, p2p
```

4.1.3 AC / Bode Analysis

The script `automation/ac_analysis.py` provides formal frequency-domain stability assessment. It reads an AC simulation output, identifies the gain crossover frequency (where magnitude crosses 0 dB), and computes the phase margin:

Phase Margin = 180° + phase_crossover

A phase margin above 45° is reported as stable. This is the most rigorous tool in the kit, grounded directly in classical control theory.

4.2 Codifying the Workflow

With tools in place, the final challenge was ensuring the agent actually used them. Left to its own devices, it would sometimes skip verification or declare success after a single simulation. Two documents codified the engineering workflow:

`Instructions.txt` defined the step-by-step methodology: parse the prompt, assign defaults for unspecified parameters, retrieve the reference netlist, calculate values, generate the netlist, verify structure, run NgSpice, inspect the waveform, and iterate if needed.

`AGENTS.md` enforced strict verification rules: the agent must never rely on a single simulation or basic averages; it must run `fft_diagnostics.py` on every output; if unstable, it must invoke `spice_optimizer.py`; and all final outputs must follow a strict naming convention in the `outputs/` directory.

These rules transformed the agent from an unconstrained reasoner into a disciplined engineering process — one that happened to be executed by an AI.

4.3 The Complete Architecture

The final system operates as a closed loop across three layers:

1. **AI Layer (Antigravity Agent).** Receives the user's specification, selects the topology, reads the library template, computes initial values, and writes the netlist. This layer handles the creative and contextual aspects — understanding what the user wants, knowing what a SEPIC converter is.
2. **Simulation Layer (NgSpice).** Executes the netlist as a deterministic, physics-based simulation. No AI is involved; it is pure numerical computation governed by Kirchhoff's laws.

3. **Diagnostic Layer (FFT + Optimizer + Bode).** Evaluates the simulation result algorithmically. If stable, the netlist is finalised. If not, the optimizer performs a grid search — entirely deterministic, no language model consulted.

The agent orchestrates these layers in a loop: synthesise, simulate, diagnose, (if unstable) optimise, re-simulate, re-diagnose, (if stable) finalise. Each iteration produces auditable artefacts: the netlist, simulation data, FFT plots, and optimizer output.

4.4 Could the Diagnostic Tools Save the LLM Pipeline?

A natural question: now that the diagnostic tools exist, could they be grafted onto the Phase 2 pipeline, restoring the LLM and dispensing with the agent?

Partially. Adding a post-simulation FFT step and the optimizer would close the feedback loop for PI tuning and would likely work for simple topologies (buck, boost).

But not fully. The Phase 2 pipeline was a fixed, linear sequence with no branching. If the optimizer failed because the instability was caused by a wrong inductor value (not wrong PI gains), the pipeline had no mechanism to diagnose the root cause and modify a different parameter. If the library manager selected the wrong template, there was no way to detect the mismatch and retry. If the LLM's calculations were off by an order of magnitude, there was no way to recognise the physically impossible waveform and start over.

The agent can do all of these things. When the optimizer sweeps the gain space and nothing works, it can reason: *"No gain combination produced a stable result. The power-stage components may be wrong. Let me recheck the inductance."* This kind of multi-level, conditional reasoning — where the failure of one tool triggers re-evaluation of upstream assumptions — is what distinguishes an agent from a pipeline.

The diagnostic tools did not make the agent unnecessary. They made it *effective*. Without them, the agent was an engineer without instruments; with them, it became an engineer with a fully equipped lab.